



CRI Bioinformatics Bootcamp 2026

Signia by Hilton La Canntera Resort & Spa
San Antonio, TX

Presented By

Katie Campbell
Nick Ceglia

Date

May 16, 2026

Course schedule



- 1 R Workshop
- 2 → Bulk RNA sequencing
- 3 → Single cell RNA sequencing
- 4 Single cell RNA sequencing (Advanced)
- 5 → Spatial transcriptomics (Xenium)
- 6 Spatial transcriptomics (Visium HD)

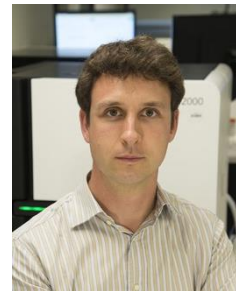
Meet the Faculty

Instructors



Washington University in St. Louis

Malachi Griffith



Washington University in St. Louis

Obi Griffith



Memorial Sloan Kettering

Nick Ceglia



Memorial Sloan Kettering

Nataly Naser Al Deen



University of California, Los Angeles

Katie Campbell

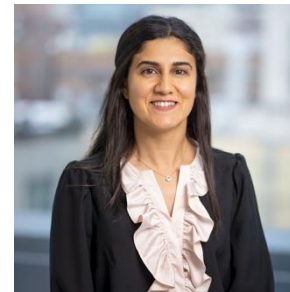
Meet the Faculty

Teaching Assistants



Stanford

Christie Chang



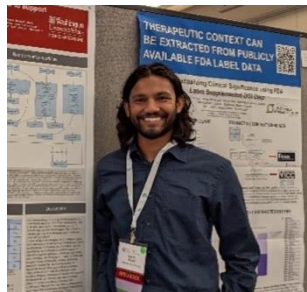
Stanford

Maryam Pourmaleki



Washington University in St. Louis

Zachary Skidmore



Washington University in St. Louis

Kartik Singhal



Washington University in St. Louis

Evelyn Schmidt

Meet the Faculty

Cancer Research Institute



Director of Scientific Affairs

Samik Upadhaya



Research Scientist

Fahad Benthani



Associate Director of Program Management

Shasell Negron

Overview

Breakdown of time

- Lectures
- Hands-on time
 - Code with us or on your own!
- Office hours
 - 1:1 sign-ups to review topics or discuss your data
- Special topics (Optional, Evening)
 - AI as a research assistant
 - Vibe coding & Agentic AI
 - Python based single cell and spatial transcriptomics
 - Python based image analysis
- Roundtable Discussions over Lunch
 2. Designing sequencing/omics experiments
 3. Career pathways in computational biology
 4. Grant planning for computational projects
 5. Code sharing and reproducibility
 6. Single-cell study design and interpretation

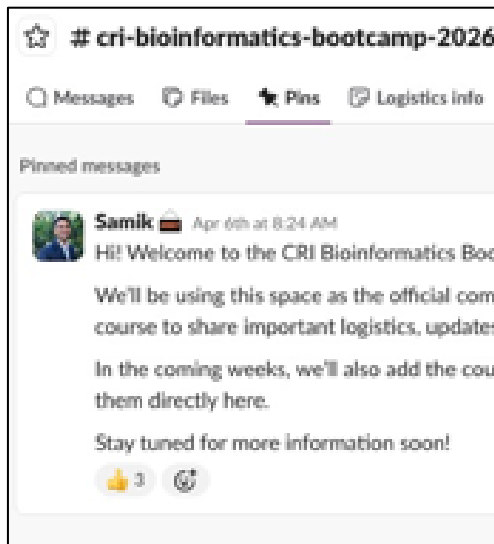
Resources during the course

<https://cancer-research-institute.github.io/>



Course materials and schedule

Website



Direct communication/Questions

Slack



Need a TA

Post-its

Communication

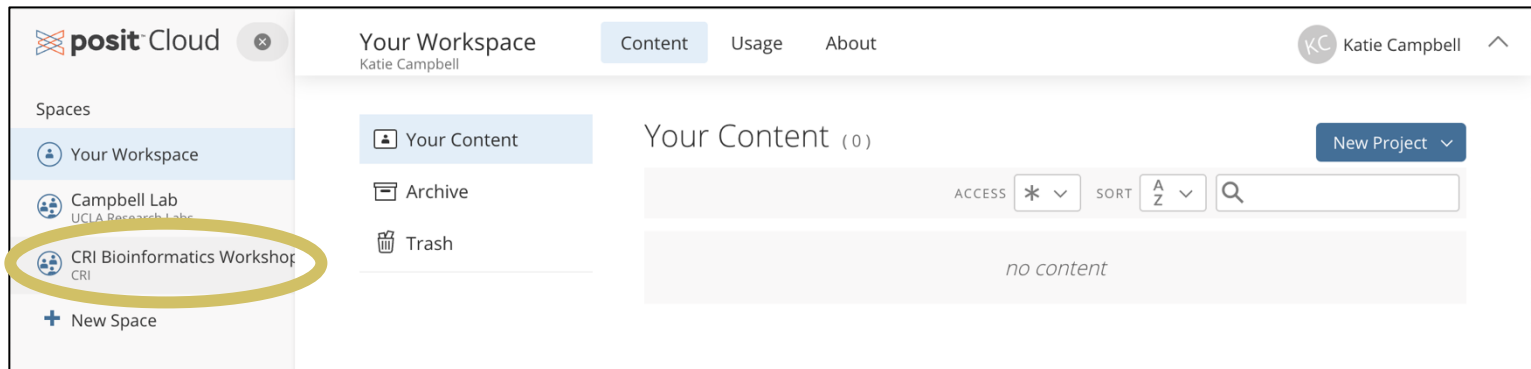
Slack

- Channel: cri-bootcamp-2026 (everyone)
 - Ask anything!
 - Tag @[Name] to call out an instructor or TA
 - Helpful hints
- Direct messages (1:1)
 - Directly message each other or faculty members

Getting started in Posit

Posit is a cloud-based server for RStudio, used during this course.

1. Log in at posit.cloud

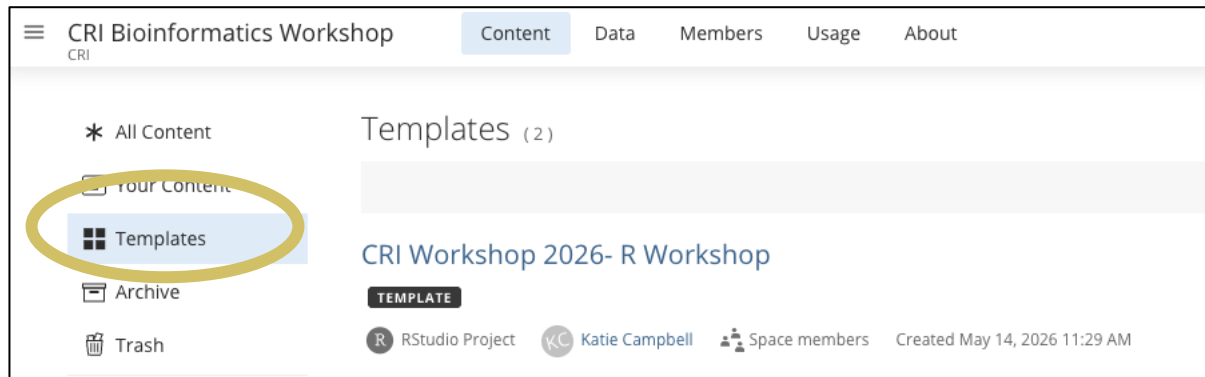


If the **CRI Bioinformatics Workshop** space is not visible on the lefthand side, slack @Samik your email address.

Templates have been created for you for the course.

The Template space has libraries installed and data uploaded for you.

2. See **Templates**

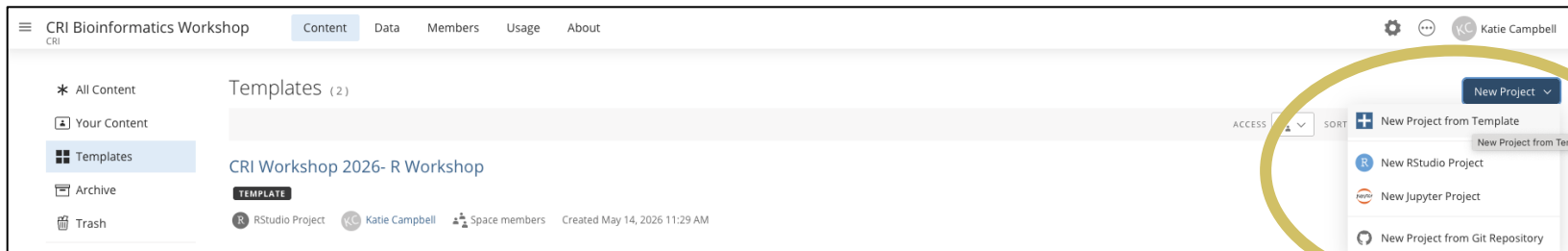


Have a question? Put a **POST-IT** on the corner of your computer screen, and a TA will be there momentarily.

Create your own Template

The Template space has libraries installed and data uploaded for you.

3. Select **New Project** → **New Project from Template**

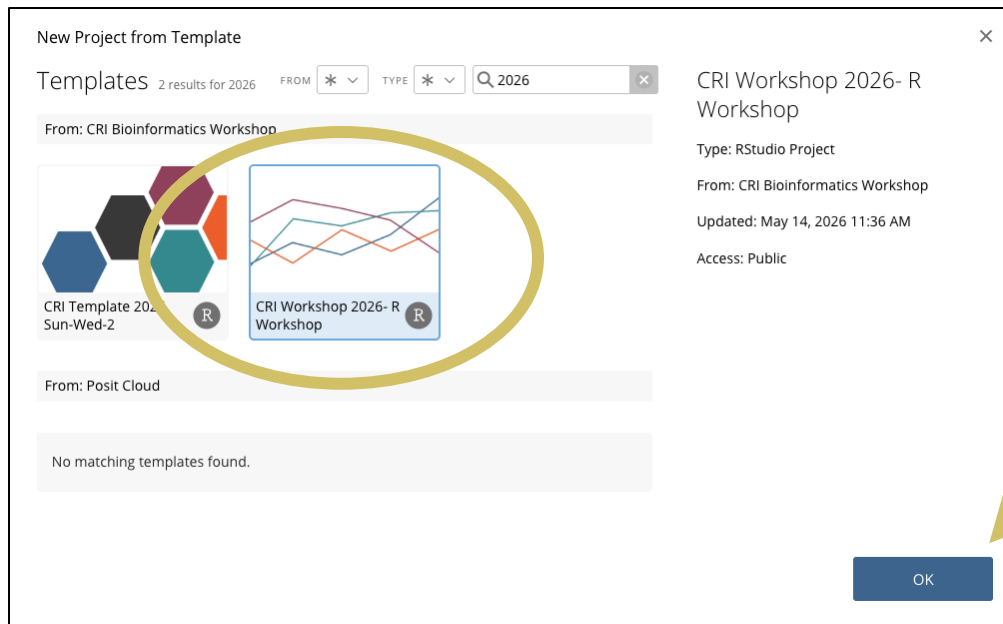


Have a question? Put a **POST-IT** on the corner of your computer screen, and a TA will be there momentarily.

Today: CRI Workshop 2026- R Workshop

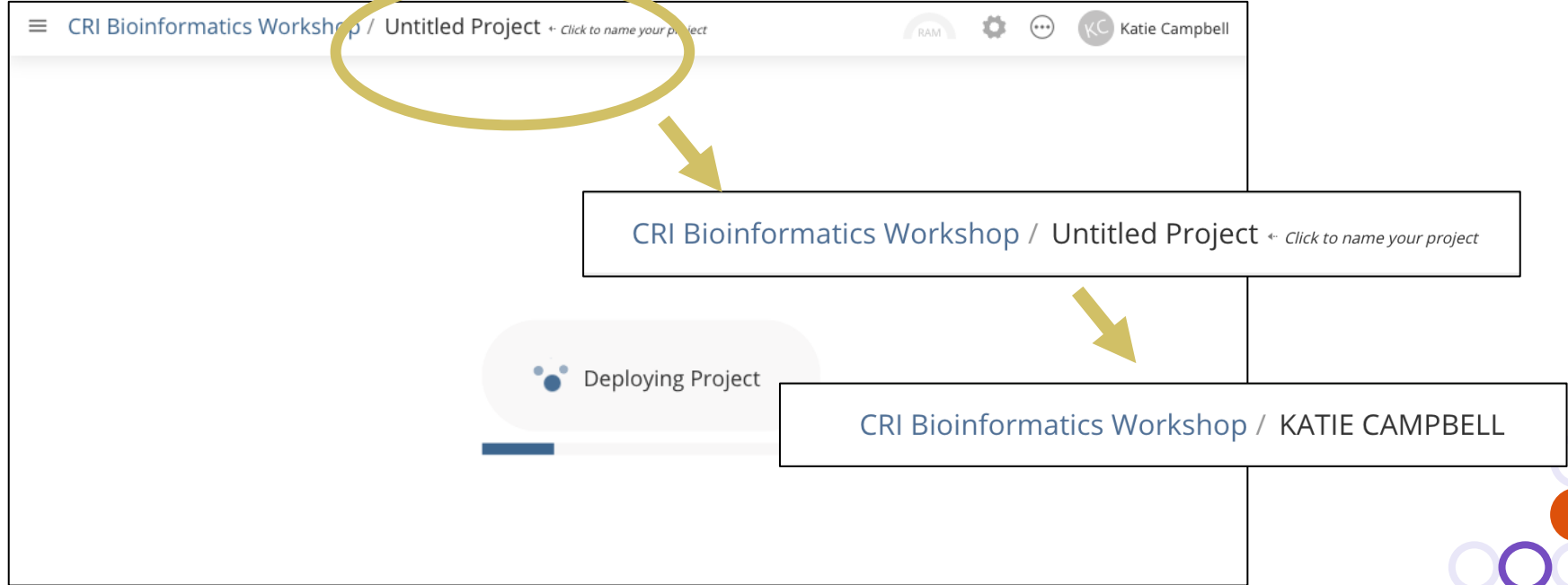
This workspace will be used for Days 1–1.5 for the R Workshop.

3. Select **CRI Workshop 2026- R Workshop** → Click **OK**

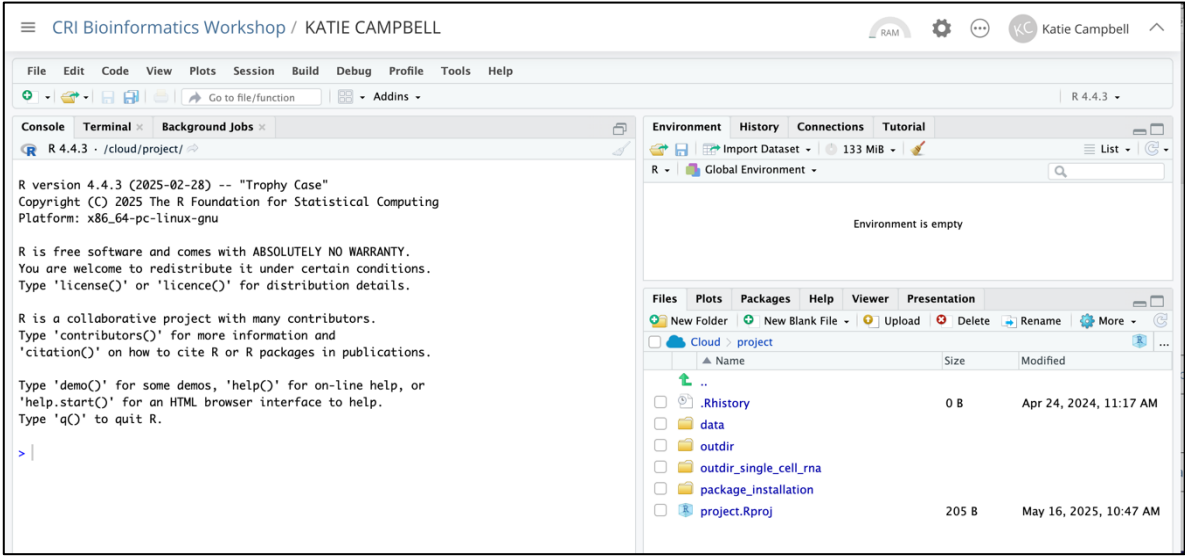


Rename your project so you can find your workspace

FIRST LAST – *You can also add “R Workshop” if you’d like.*



Only use YOUR workspaces for the course



The screenshot displays the RStudio interface within a web browser. The top bar indicates the user is logged in as 'Katie Campbell' and is in the 'CRI Bioinformatics Workshop' workspace. The interface is divided into several panes:

- Console:** Shows the R version (4.4.3) and the current directory (/cloud/project). It displays the R startup message, including the copyright notice for 2025 and the license information.
- Environment:** Shows the 'Global Environment' with the message 'Environment is empty'.
- Files:** A file explorer showing the contents of the 'project' directory. It lists several files and folders, including .Rhistory, data, outdir, outdir_single_cell_rna, package_installation, and project.Rproj.

The file explorer table is as follows:

Name	Size	Modified
..		
.Rhistory	0 B	Apr 24, 2024, 11:17 AM
data		
outdir		
outdir_single_cell_rna		
package_installation		
project.Rproj	205 B	May 16, 2025, 10:47 AM

R Workshop

Introduction to R and RStudio

Presented By

Nick Ceglia
Katie Campbell

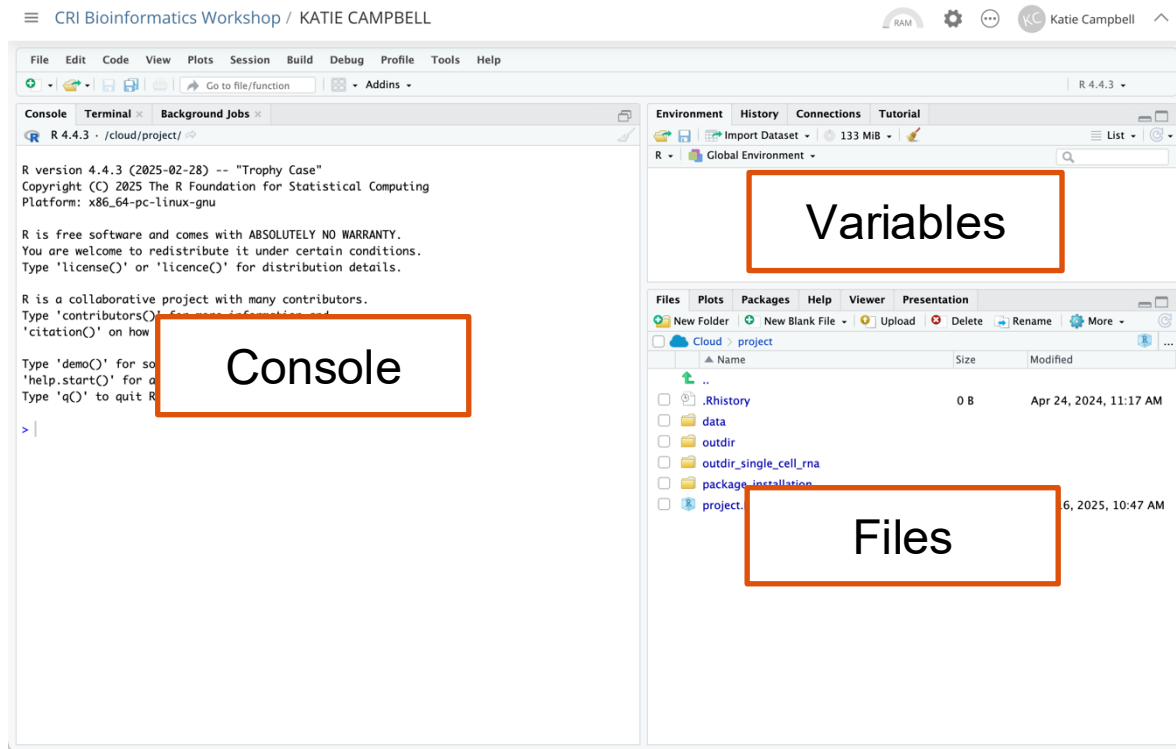
Date

May 16, 2026

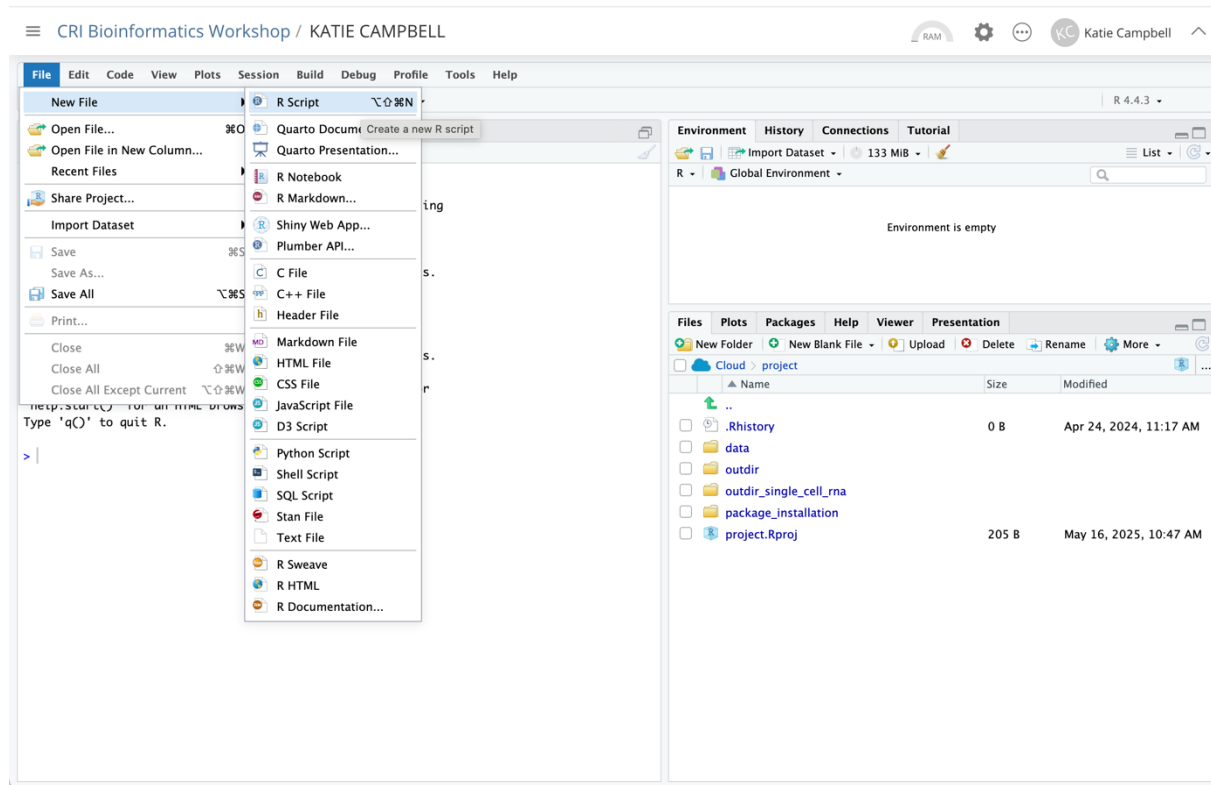
Goals for the R Workshop

- Feel comfortable reading and running commands
- Figure out how to figure things out
- Learn enough to be dangerous

RStudio Interface



Creating a script

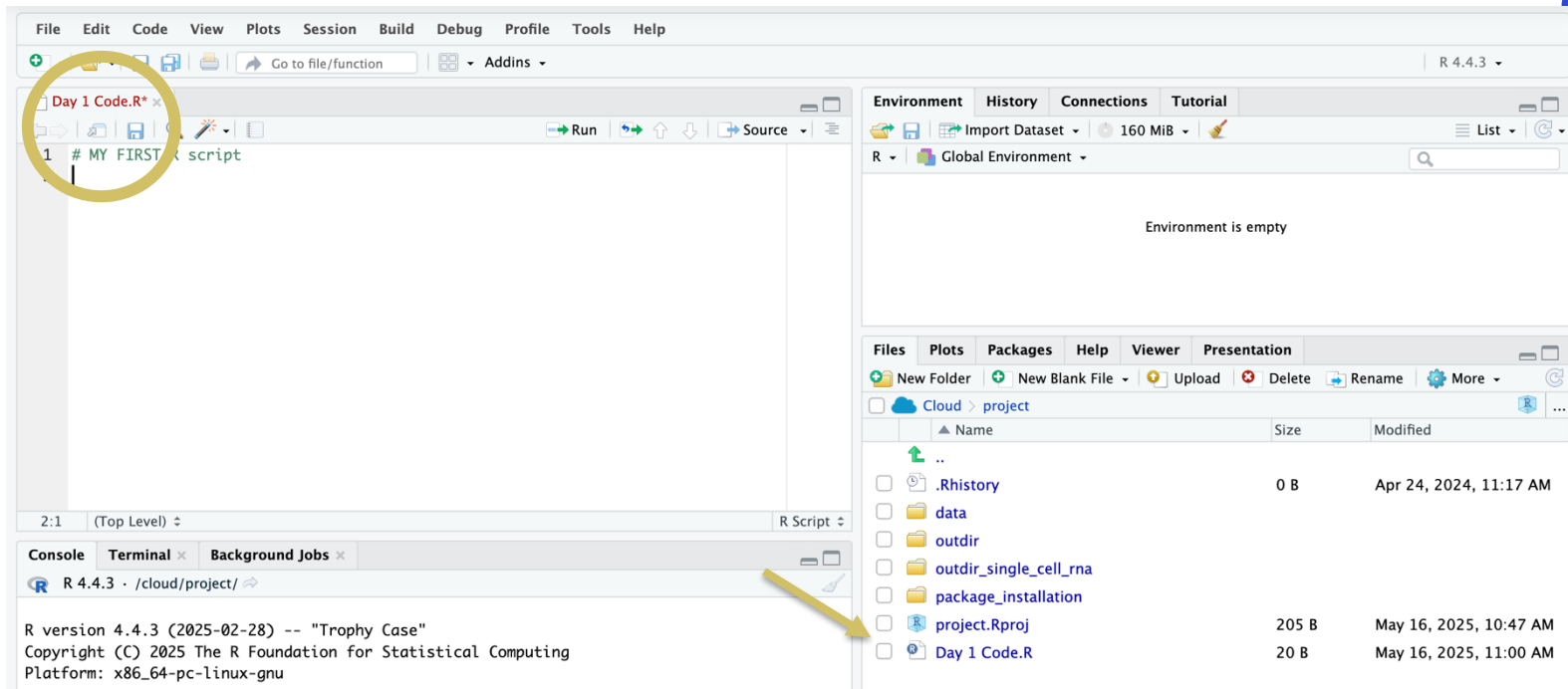


Saving code in the script vs coding in the console

Code saved in your workspace will not be available to others.

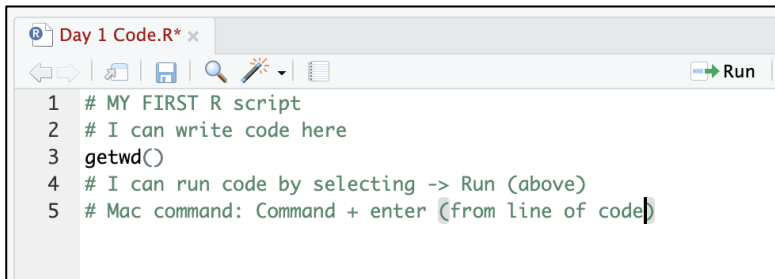
- Script
 - Anything you want to come back to later
 - Storing variables
 - Saving progress
- Console
 - Evaluating sizes, shapes, and summaries of data types
 - Listing
 - Testing code

Saving your first RScript



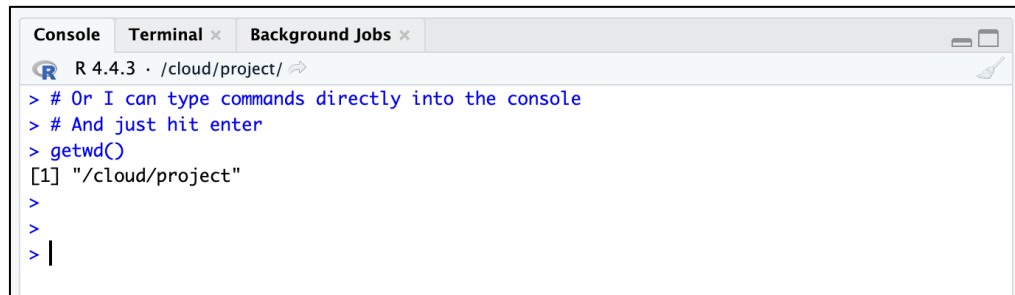
Running pieces of code

From a script



```
1 # MY FIRST R script
2 # I can write code here
3 getwd()
4 # I can run code by selecting -> Run (above)
5 # Mac command: Command + enter (from line of code)
```

From the console



```
Console Terminal Background Jobs
R 4.4.3 · /cloud/project/
> # Or I can type commands directly into the console
> # And just hit enter
> getwd()
[1] "/cloud/project"
>
>
> |
```

The working directory: Orienting your filesystem

```
# Get the current working directory
current_dir <- getwd()
print(paste("Current directory:", current_dir))

# List files in the current directory
files <- list.files()
print("Files in the current directory:")
print(files)

# Change the current directory
new_dir <- "path/to/new/directory"
setwd(new_dir)
print(paste("Changed directory to:", new_dir))
```

The working directory: Orienting the filesystem of your computer



```
# Get the current working directory
current_dir <- getwd()
print(paste("Current directory:", current_dir))

# List files in the current directory
files <- list.files()
print("Files in the current directory:")
print(files)

# Change the current directory
new_dir <- "path/to/new/directory"
setwd(new_dir)
print(paste("Changed directory to:", new_dir))
```

Functions are indicated by
function_name(parameters)

paste() can be used to
concatenate strings together

print() will print out what's inside
the parentheses into the console

<- assigns the content on the right
(the string) to the variable name
on the left



Assigning variables saves them in your global environment

The screenshot displays the RStudio interface with the following components:

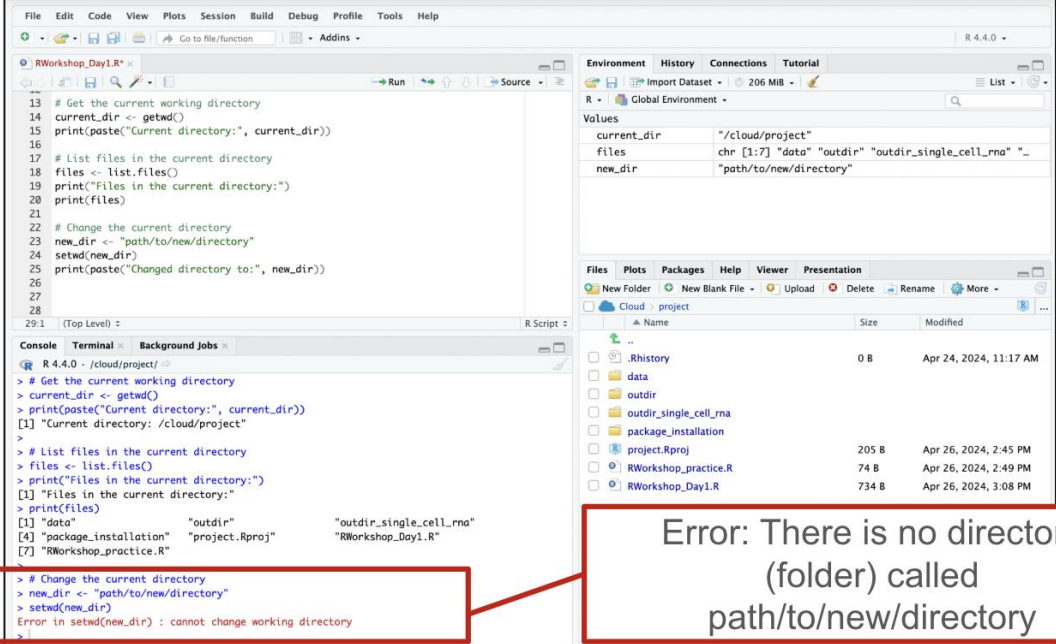
- Source Editor:** Contains R code for getting the current directory, listing files, and changing to a new directory. Red circles highlight the `current_dir <- getwd()`, `files <- list.files()`, and `new_dir <- "path/to/new/directory"` lines. Red arrows point from these circles to the Environment pane.
- Environment Pane:** Shows the global environment with three variables:

Variable	Value
<code>current_dir</code>	<code>"/cloud/project"</code>
<code>files</code>	<code>chr [1:7] "data" "outdir" "outdir_single_cell_rna" ..</code>
<code>new_dir</code>	<code>"path/to/new/directory"</code>
- Console:** Shows the output of the code execution. Red boxes highlight the output of `list.files()` and the error message for `setwd(new_dir)`.

```
> # List files in the current directory
> files <- list.files()
> print("Files in the current directory:")
[1] "Files in the current directory:"
> print(files)
[1] "data"          "outdir"        "outdir_single_cell_rna"
[4] "package_installation" "project.Rproj" "RWorkshop_Day1.R"
[7] "RWorkshop_practice.R"

> # Change the current directory
> new_dir <- "path/to/new/directory"
> setwd(new_dir)
Error in setwd(new_dir) : cannot change working directory
> |
```
- Files Pane:** Shows the file structure of the project. A red box highlights the files `data`, `outdir`, `outdir_single_cell_rna`, `package_installation`, `project.Rproj`, `RWorkshop_practice.R`, and `RWorkshop_Day1.R`.

Be prepared for Errors...



The screenshot shows the RStudio interface with the following components:

- Source Editor:** Contains R code for getting the current directory, listing files, and attempting to change to a new directory.
- Environment:** Shows variables: `current_dir` (character), `files` (character vector), and `new_dir` (character).
- Files:** A file explorer showing the project structure with folders like `data`, `outdir`, and `outdir_single_cell_rna`.
- Console:** Displays the execution output and an error message.

Code in Source Editor:

```
13 # Get the current working directory
14 current_dir <- getwd()
15 print(paste("Current directory:", current_dir))
16
17 # List files in the current directory
18 files <- list.files()
19 print("Files in the current directory:")
20 print(files)
21
22 # Change the current directory
23 new_dir <- "path/to/new/directory"
24 setwd(new_dir)
25 print(paste("Changed directory to:", new_dir))
26
27
28
29.1 (Top Level) :
```

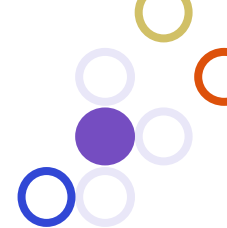
Console Output:

```
R 4.4.0 - /cloud/project/
> # Get the current working directory
> current_dir <- getwd()
> print(paste("Current directory:", current_dir))
[1] "Current directory: /cloud/project"
>
> # List files in the current directory
> files <- list.files()
> print("Files in the current directory:")
[1] "Files in the current directory:"
> print(files)
[1] "data"          "outdir"        "outdir_single_cell_rna"
[4] "package_installation" "project.Rproj" "RWorkshop_Day1.R"
[7] "RWorkshop_practice.R"
>
> # Change the current directory
> new_dir <- "path/to/new/directory"
> setwd(new_dir)
Error in setwd(new_dir) : cannot change working directory
> |
```

Error Message:

Error: There is no directory (folder) called path/to/new/directory

Using packages that have already built functions for you



- Base R: Functions that already exist
- Common error: "Could not find function"
 - Install the package that has the function
 - Load the package that has the function

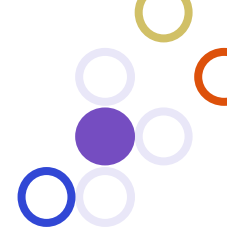
```
> mutate()  
Error in mutate() : could not find function "mutate"
```

```
> install.packages("dplyr")  
> library(dplyr)
```

*You only need to install packages once (unless you are updating).
All of the packages you need for this course are already installed.*



Primitive variables in R



- **Most basic data types**
 - Cannot be decomposed
- **Numeric (Float)**
 - Default variable for numbers
 - Includes floating point and integer
- **Integer**
- **Logical (Boolean)**
 - TRUE or FALSE
 - Conditional Statements
 - And/Or/Not
- **Character (String)**
 - Text data
 - "Hello world!"
- **Other types**
 - Raw (Byte Representation)
 - Complex
- **Data structures**
 - Composite of primitive

Numeric

```
# Numeric variable
num_var <- 10
print(num_var) # Output: 10

# Arithmetic operations
result <- num_var * 2
print(result) # Output: 20
```

Integer

```
# Integer variable
int_var <- 20L # The 'L' suffix indicates an integer
print(int_var) # Output: 20

# Integer arithmetic
result <- int_var / 5
print(result) # Output: 4
```

Character

```
# Character variable
char_var <- "Hello, World!"
print(char_var) # Output: Hello, World!

# Concatenation
new_string <- paste(char_var, "This is R programming.")
print(new_string) # Output: Hello, World! This is R programming.
```



Vectors and Lists

- **Vector**

- 1 dimensional
- Single data type
 - Numeric, Character, Logical
- Efficient (pre-allocation)
- Building Matrices

- **List**

- Heterogeneous data
- Dynamic
- Creating Dataframes

- **Factor**

- Categorical data
- Levels
- Memory efficient
 - Fixed set of values can be mapped with an integer

Vector

```
# Creating a numeric vector
numeric_vector <- c(1, 2, 5.3, 6, -2, 4)

# Creating a character vector
character_vector <- c("apple", "banana", "cherry")

# Creating a logical vector
logical_vector <- c(TRUE, FALSE, TRUE, FALSE)

#Pre-allocate
my_vector <- numeric(10)
```

List

```
# Creating a list with mixed types
my_list <- list(
  barcode = "AAAGCAAGTTACGCGC",
  umis = 25,
  seq = c("CASSLVLPGYTEAFF", "CASVGQGEYEYF"),
  cell = TRUE,
)

# Accessing elements
print(my_list$barcode)      # Outputs "AAAGCAAGTTACGCGC"
print(my_list$seq)         # Outputs c("CASSLVLPGYTEAFF", "CASVGQGEYEYF")
```

Matrix

- 2 dimensional
- Single data type
- *matrix()*
 - data
 - nrow
 - ncol
 - byrow
 - Whether to fill by rows or columns
 - dimnames'
- Access elements by row and column
- Matrix operations

Creating a Matrix:

```
# Create a matrix with random values
mat <- matrix(rnorm(20), nrow = 4, ncol = 5)
```

Matrix operations

```
element <- mat[1, 2]

# Calculate row sums
row_sums <- rowSums(mat)

# Calculate column sums
col_sums <- colSums(mat)

# Create another matrix
mat2 <- matrix(rnorm(20), nrow = 5, ncol = 4)

# Perform matrix multiplication
mat_mult <- mat %*% mat2

# Transpose the matrix
mat_transpose <- t(mat)

# Select the first two rows
first_two_rows <- mat[1:2, ]

# Select the first three columns
first_three_cols <- mat[, 1:3]
```

Dataframe

- **What is a dataframe?**
 - "Excel"-like structure
 - Heterogeneous data
 - Labeled columns and rows
 - `data.frame()`
- **Using the dataframe**
 - Accessing columns by name
 - `df$column_name`
 - Access by row and column
 - `df[i,j]`
 - Filtering
 - `subset(df, column_name > 25)`
 - Add columns
 - `df$col2 <- c(1,2,3)`
 - Add rows
 - `df <- rbind(df, c("w",2,True))`
 - Modify values
 - `df[i,j] <- 25`

```
# Create a data.frame
df <- data.frame(
  barcode = c("AAACCTGAGAAGGCCT-1", "AAACCTGCAAGTTGTC-1", "AAACCTGCACCGATAT-1"),
  phenotype = c("TEMRA", "Naive", "Dysfunctional"),
  treatment = c("Pre", "Post", "Pre"),
  response = c("R", "NR", "R")
)

result <- df$phenotype
print(result) # prints "TEMRA","Naive","Dysfunctional"

result <- df[1,2]
print(result) # prints "TEMRA"

result <- subset(df,treatment=="Post")
print(result) # prints AACCTGCAAGTTGTC-1 Naive Post NR

df$site <- c("Tumor","Tumor","Blood")

df <- rbind(df,c("AAACCTGCACCGATAT-1","Naive","Post","R","Tumoxr"))

df[4,5] <- "Tumor"
```

```
> df
```

	barcode	phenotype	treatment	response	site
1	AAACCTGAGAAGGCCT-1	TEMRA	Pre	R	Tumor
2	AAACCTGCAAGTTGTC-1	Naive	Post	NR	Tumor
3	AAACCTGCACCGATAT-1	Dysfunctional	Pre	R	Blood
4	AAACCTGCACCGATAT-1	Naive	Post	R	Tumor

String manipulation

- Strings vs substrings
 - Substring is a sequence of text within a string.
 - "Hello" is a substring of "Hello world!"
- Case conversion
 - Uppercase and lower matter, R is case-sensitive
- Find and replace
 - Find a substring and replace it with another substring
 - Examples: ID matching, motifs, etc.
- Length calculation
 - Sequence length
- Stringr package (tidyverse)
 - str_length
 - str_to_upper
 - str_locate
 - str_sub
 - str_replace and str_replace_all
- Base: grep, grepl, gsub
- Regular expressions
 - Search pattern
 - "hell*old!" matches "hello world!"

Length Calculation

Determining the length of sequences.

```
sequence_length <- str_length(sequence)
print(sequence_length)
```

Case Conversion

Converting uppercase to lowercase, or vice versa.

```
sequence_upper <- str_to_upper(sequence)
print(sequence_upper)
```

Substring Extraction

Extracting parts of sequences, such as cutting out genes or regions of interest.

```
extracted_sequence <- str_sub(sequence, 3, 8)
print(extracted_sequence)
```

Replacing Substrings

Modifying sequences by replacing specific nucleotides or amino acids.

```
dna_sequence <- "ATGCGTACGTTGACT"
rna_sequence <- str_replace_all(dna_sequence, "T", "U")
print(rna_sequence)
```

Logical operations

- Evaluate to True or False
 - logical data type
- Use case
 - Conditional statements
 - if / else
 - Filtering data
 - `subset(df,treatment="post")`
- Comparisons
 - Equal: `==`
 - Not equal: `!=`
 - Greater Than: `">"`
 - Less Than `"<"`
 - `%in%`
 - Test inclusion of one vector in another
- Boolean statements
 - and: both are true `"&"`
 - or: atleast one is true `"|"`
 - not: evaluates to false `"!"`

```
# Greater than
result_gt <- x > y
print(result_gt) # Output: FALSE

# Less than
result_lt <- x < y
print(result_lt) # Output: TRUE

# Equal to
result_eq <- x == y
print(result_eq) # Output: FALSE

# Not equal to
result_neq <- x != y
print(result_neq) # Output: TRUE
```

```
x <- 3
if (x > 5) {
  print("x is greater than 5")
} else if (x == 5) {
  print("x is equal to 5")
} else {
  print("x is less than 5")
}
```

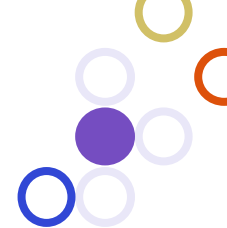
```
# Logical operations
a <- TRUE
b <- FALSE

# AND operation
result_and <- a & b
print(result_and) # Output: FALSE

# OR operation
result_or <- a | b
print(result_or) # Output: TRUE

# NOT operation
result_not <- !a
print(result_not) # Output: FALSE
```


Writing your own functions



- Encapsulate reusable block of code for a specific task

- Elements of a function

- Name
- Input (Arguments)
- Body
- Output (Optional)
 - Writing to a file
 - Plotting

```
function_name <- function(argument1, argument2, ...) {  
  # Function body  
  # Perform operations  
  # Return a value (optional)  
}
```

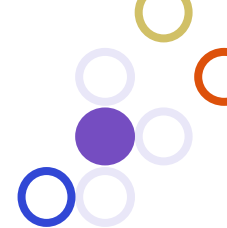
- Why write functions?

- Breaking down larger problems
- Reusability
- Minimizing errors
- Maintainability

```
# Define a function to calculate the square of a number  
square <- function(x) {  
  return(x^2)  
}  
  
# Call the function  
result <- square(5)  
print(result) # Output: 25
```



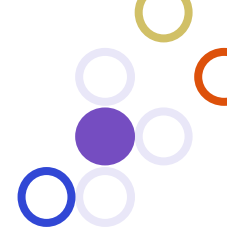
Manipulating data frames



- **What “Tidy” means:**
 - Each variable is a column
 - Each observation is a row
 - A table is a consistent set of observations for a known set of variables.
- **Tidyverse Packages for Data Manipulation:**
 - **dplyr:** Provides a grammar of data manipulation for data frames, enabling easy filtering, selecting, mutating, summarizing, and arranging of data.
 - **tidyr:** Offers tools for reshaping and tidying messy datasets, such as `gather()` and `spread()` functions for converting between wide and long formats.
 - **ggplot2:** Allows for the creation of sophisticated data visualizations using a layered grammar of graphics.



Manipulating data frames

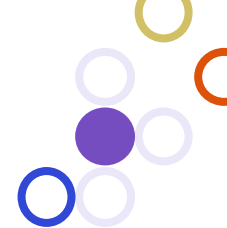


- Nice features:
 - Printing
 - Variable information
 - Extracting a column for a tibble yields another tibble
 - data.frame will yield a vector
 - Row index not added by default
 - Explicit id column

```
library(tidyverse)

# Create a tibble (tidyverse version of data.frame)
df <- tibble(
  barcode = c("AAACCTGAGAAGGCCT-1", "AAACCTGCAAGTTGTC-1", "AAACCTGCACCGATAT-1"),
  phenotype = c("TEMRA", "Naive", "Dysfunctional"),
  treatment = c("Pre", "Post", "Pre"),
  response = c("R", "NR", "R")
)
```

Hands-on: Reading, writing, and interpreting data structures



1. Start your first Rscript
2. Work through the blocks of code under the **Course: Introduction to R** page
 - Review the details on how the code works in the Lecture slides for assistance
 - Put a post-it on your laptop if you get stuck, indicating for a TA to come up to you
 - Work through the blocks of code on this page, practicing in both your Rscript and the console
3. Take the next step
 - There are a list of **Additional exercises** at the bottom of the page for you to try on your own



Goal: Start building your R vocabulary

Are your hands as smart as your brain?

Presented By

Nick Ceglia
Katie Campbell

Date

May 16, 2026

R Workshop

Introduction to plotting and statistics

Presented By

Nick Ceglia
Katie Campbell

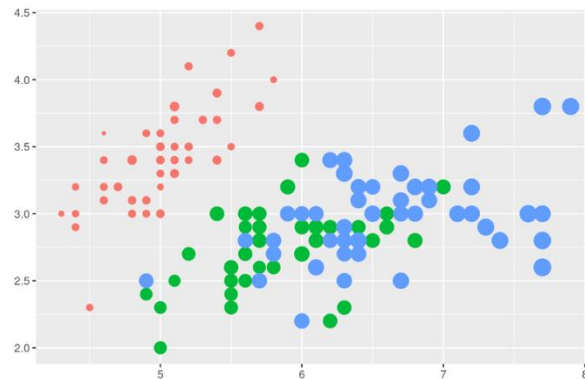
Date

May 16, 2026

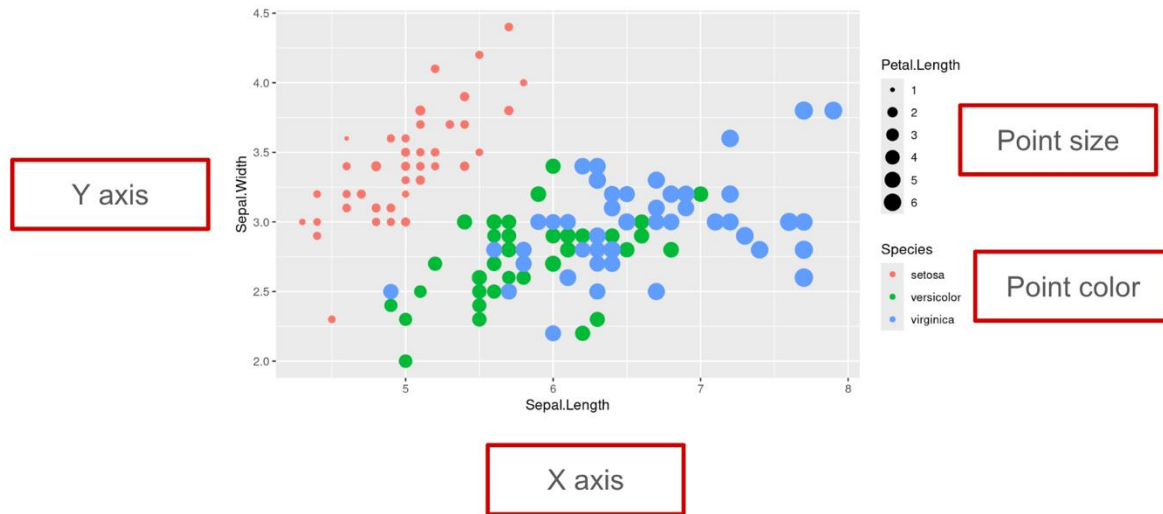
Introduction to plotting and statistics

Hands-on: Plotting information from data structures for real-time analysis

Student	Subject	Score
A	Math	99
A	Literature	45
A	PE	56
B	Math	73

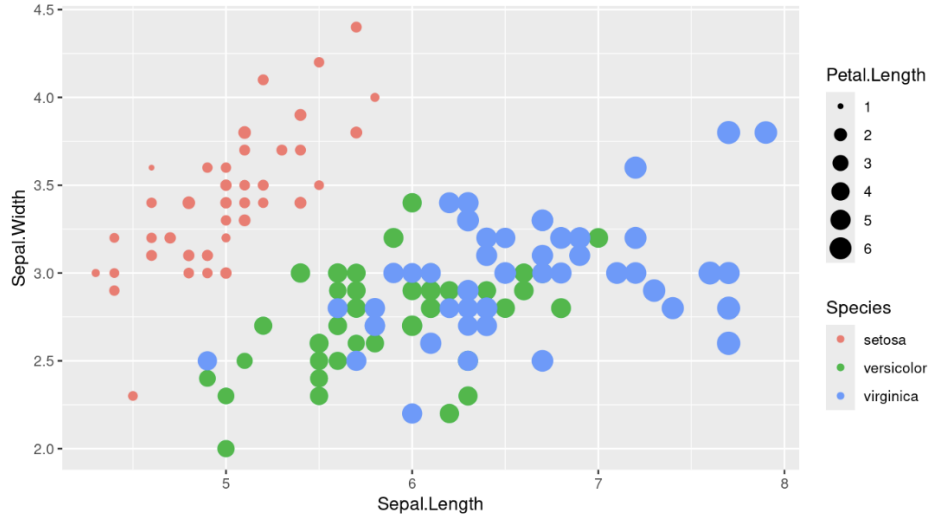


Variables are individually represented in the dimensions of a plot



Plot dimensions motivate data interpretation and analysis

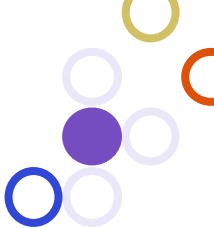
To
va



“The pink points are smaller and are higher/to the left of the bigger blue/green points.”

“The **setosa** species has shorter petal lengths, and greater sepal width and sepal length, compared to the **versicolor** and **virginica** species.”

Data generally comes in “wide” format



- Can be easier to understand
- More convenient for certain analyses or visualization
- No information is repeated in each line

Example of wide format:

Student	Math	Literature	PE
A	99	45	56
B	73	78	55
C	12	96	57



Long data format is the “tidy” format

- Multiple rows correspond to a single observation and measurement
- One column for each variable
- Additional columns store metadata or grouping variables

Example of long format:

Student	Subject	Score
A	Math	99
A	Literature	45
A	PE	56
B	Math	73
B	Literature	78
B	PE	55
C	Math	12

Manipulating data frames: Long to wide data

```
library(tidyr)

# Example long format data
long_data <- data.frame(
  Subject = c("A", "A", "B", "B"),
  Time = c(1, 2, 1, 2),
  Measurement = c(10, 15, 12, 18)
)

# Convert long format data to wide format
wide_data <- spread(long_data, key = Time, value = Measurement)

# View the wide format data
print(wide_data)
```

```
> head(long_data)
  Subject Time Measurement
1      A    1         10
2      A    2         15
3      B    1         12
4      B    2         18
```

```
> wide_data
  Subject   1   2
1      A  10  15
2      B  12  18
```

Now `1` and `2` can be directly compared

Manipulating data frames: Wide to long data

```
library(tidyr)

# Example wide format data
wide_data <- data.frame(
  Subject = c("A", "B"),
  Time1 = c(10, 12),
  Time2 = c(15, 18)
)

# Convert wide format data to long format
long_data <- gather(wide_data, key = Time, value = Measurement, -Subject)

# View the long format data
print(long_data)
```

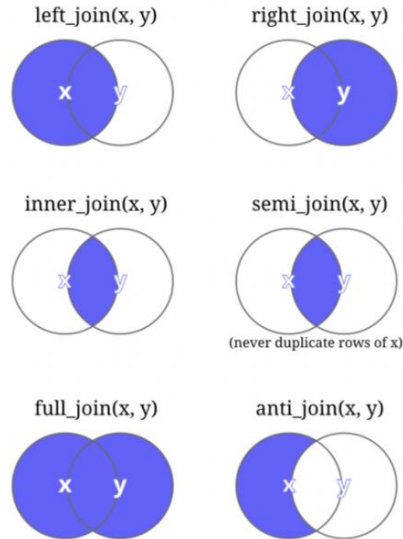
```
> wide_data
  Subject Time1 Time2
1      A    10    15
2      B    12    18
>
>
> long_data
  Subject Time Measurement
1      A     1         10
2      A     2         15
3      B     1         12
4      B     2         18
```

Now `Time` and `Measurement`
can be directly compared

Merging data

What if you want to compare data from two different data frames?

dplyr *joins*



```
library(dplyr)

# Example datasets
df1 <- data.frame(ID = c(1, 2, 3), Name = c("Alice", "Bob", "Charlie"))
df2 <- data.frame(ID = c(2, 3, 4), Score = c(85, 90, 95))

# Inner join
inner_merged <- inner_join(df1, df2, by = "ID")

# Left join
left_merged <- left_join(df1, df2, by = "ID")

# Right join
right_merged <- right_join(df1, df2, by = "ID")

# Full join
full_merged <- full_join(df1, df2, by = "ID")

# Semi-join
semi_merged <- semi_join(df1, df2, by = "ID")

# Anti-join
anti_merged <- anti_join(df1, df2, by = "ID")
```

*_join() functions

combine the information across two data frames

```
> full_merged
```

	ID	Name	Score
1	1	Alice	NA
2	2	Bob	85
3	3	Charlie	90
4	4	<NA>	95

```
library(dplyr)

# Example datasets
df1 <- data.frame(ID = c(1, 2, 3), Name = c("Alice", "Bob", "Charlie"))
df2 <- data.frame(ID = c(2, 3, 4), Score = c(85, 90, 95))

# Inner join
inner_merged <- inner_join(df1, df2, by = "ID")

# Left join
left_merged <- left_join(df1, df2, by = "ID")

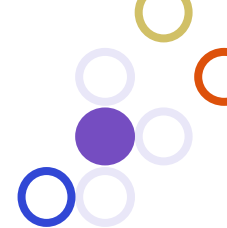
# Right join
right_merged <- right_join(df1, df2, by = "ID")

# Full join
full_merged <- full_join(df1, df2, by = "ID")

# Semi-join
semi_merged <- semi_join(df1, df2, by = "ID")

# Anti-join
anti_merged <- anti_join(df1, df2, by = "ID")
```

Using the ggplot2 package: Structuring a plot command



- Data
- Aesthetics (aes)
- Geometries (geom)
- Scales
- Coordinate systems
- Facets
- Themes

```
library(ggplot2)

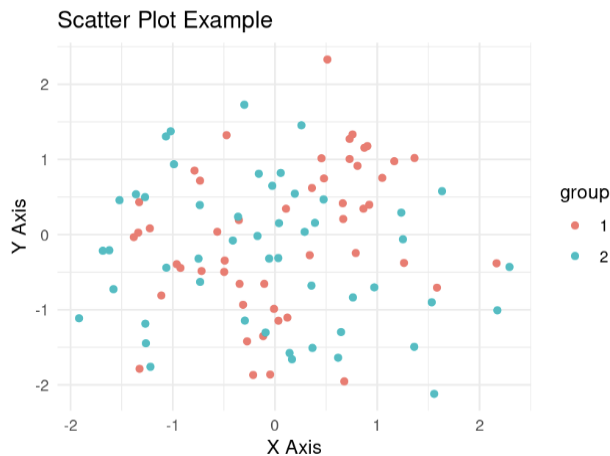
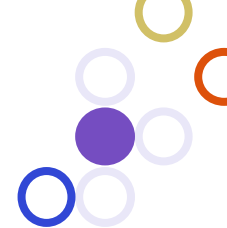
# Sample data
df <- data.frame(
  x = rnorm(100),
  y = rnorm(100),
  group = factor(rep(1:2, each = 50))
)

# Creating a scatter plot
ggplot(df, aes(x = x, y = y, color = group)) +
  geom_point() +
  theme_minimal() +
  labs(title = "Scatter Plot Example", x = "X Axis", y = "Y Axis")
```



<https://ggplot2.tidyverse.org/reference/>

Using the ggplot2 package: Structuring a plot command



```
library(ggplot2)

# Sample data
df <- data.frame(
  x = rnorm(100),
  y = rnorm(100),
  group = factor(rep(1:2, each = 50))
)

# Creating a scatter plot
ggplot(df, aes(x = x, y = y, color = group)) +
  geom_point() +
  theme_minimal() +
  labs(title = "Scatter Plot Example", x = "X Axis", y = "Y Axis")
```

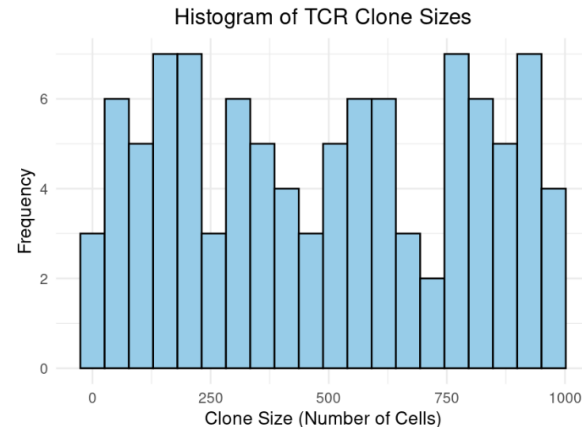
Using the ggplot2 package: Structuring a plot command

```
library(ggplot2)
library(dplyr)

# Step 1: Simulate data
set.seed(123) # Set seed for reproducibility
num_clonotypes <- 100 # Specify the number of different clonotypes

# Create a data frame with random cell counts for each clonotype
tcr_data <- tibble(
  clonotype = paste("TCR", seq_len(num_clonotypes), sep=""),
  cell_count = sample(1:1000, num_clonotypes, replace=TRUE) # Random cell
)

# Step 2: Create a histogram using ggplot2
ggplot(tcr_data, aes(x = cell_count)) +
  geom_histogram(bins = 20, fill = "skyblue", color = "black") +
  theme_minimal() +
  labs(
    title = "Histogram of TCR Clone Sizes",
    x = "Clone Size (Number of Cells)",
    y = "Frequency"
  ) +
  theme(
    plot.title = element_text(hjust = 0.5) # Center the plot title
  )
```



Performing statistics

Most statistical test commands are just their names (e.g. wilcox)

```
# Randomly generated sample data: Immune marker levels in two patient groups
group1 <- rnorm(30, mean = 5, sd = 1.5) # Patients with a mutation
group2 <- rnorm(30, mean = 4.5, sd = 1.2) # Patients without the mutation

# Perform a t-test
test <- t.test(group1, group2)

# Print the result
print(test)
```

```
> print(test)
```

Welch Two Sample t-test

data: group1 and group2

t = 1.2838, df = 57.999, p-value = 0.2043

alternative hypothesis: true difference in means is not equal to 0

95 percent confidence interval:

-0.2731337 1.2498937

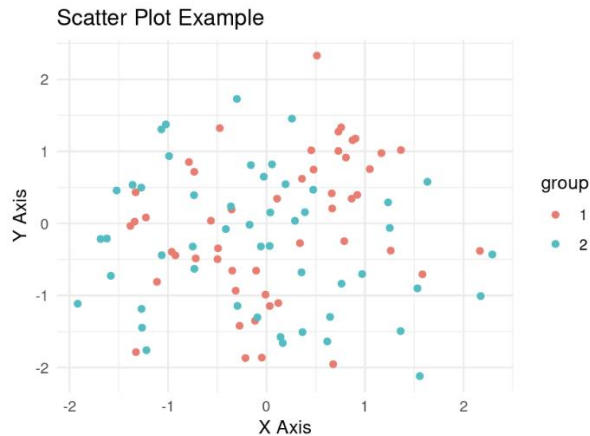
sample estimates:

mean of x mean of y

5.23333 4.74495

Is the visualization appropriate for the comparison?

I can't see the difference between groups 1 and 2...

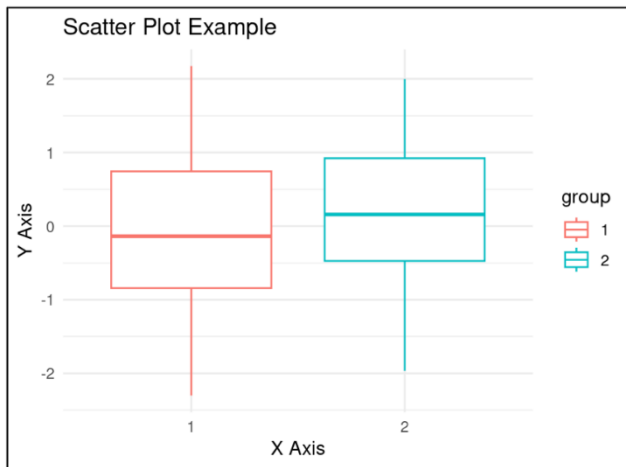
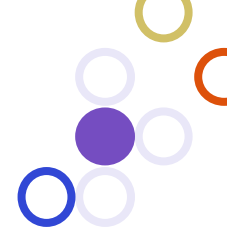


```
library(ggplot2)

# Sample data
df <- data.frame(
  x = rnorm(100),
  y = rnorm(100),
  group = factor(rep(1:2, each = 50))
)

# Creating a scatter plot
ggplot(df, aes(x = x, y = y, color = group)) +
  geom_point() +
  theme_minimal() +
  labs(title = "Scatter Plot Example", x = "X Axis", y = "Y Axis")
```

Consider different types of plots

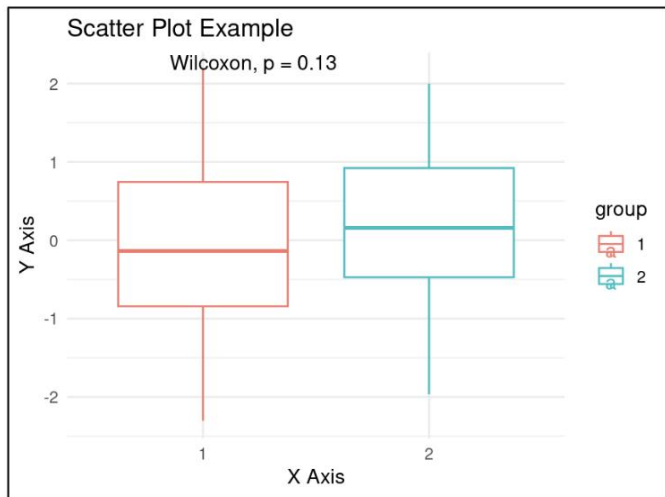


```
# Creating a box plot
ggplot(df, aes(x = group, y = y, color = group)) +
  geom_boxplot() +
  theme_minimal() +
  labs(title = "Scatter Plot Example", x = "X Axis", y = "Y Axis")
```



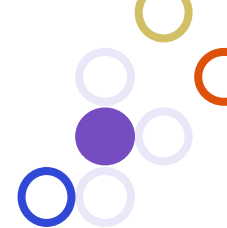
Adding a Wilcoxon test result to the plot

Using the ggpubr package



```
library(ggpubr)
# Creating a box plot
ggplot(df, aes(x = group, y = y, color = group)) +
  geom_boxplot() +
  stat_compare_means(method = 'wilcox') +
  theme_minimal() +
  labs(title = "Scatter Plot Example", x = "X Axis", y = "Y Axis")
```

Hands-on: Reading, writing, and interpreting data structures



1. Consider starting and saving a new RScript
2. Work through the blocks of code under the **Course: Basic plotting and statistics** page
 - Review the details on how the code works in the Lecture slides for assistance
 - Put a post-it on your laptop if you get stuck, indicating for a TA to come up to you
 - Work through the blocks of code on this page, practicing in both your Rscript and the console
3. Take the next step
 - There are a list of **Additional exercises** at the bottom of the page for you to try on your own



Goal: Design your first plot

How would you visualize one of your datasets?

Presented By

Nick Ceglia
Katie Campbell

Date

May 16, 2026